Тема 6. Типы данных нелинейной структуры

Лекции 15

Деревья

Цели и задачи изучения темы

Целью данной темы является изучение типов данных нелинейной структуры, таких как графы и деревья. В ней рассматриваются основные понятия и определения теории графов, способы представления графов в памяти ЭВМ, алгоритмы обхода графов, основные понятия и определения деревьев, широко распространенный частный случай деревьев – двоичные деревья и их применение, структуры данных для представления и операции над двоичными деревьями.

6.4. Деревья. Основные определения

Как уже отмечалось, дерево является частным случаем графа, в котором отсутствуют циклы. Другое определение: *дерево* – это совокупность элементов и отношений, которые образуют иерархическую структуру. Элементы дерева называются узлами. Древовидная структура имеет множество узлов, происходящих от единственного начального узла, который называется корнем. На рис. 6.9 приведен пример древовидной структуры.

Корень дерева – это такой узел, который не имеет предков (в нашем примере это узел *A*). Каждое дерево может иметь только один корень.

Самые нижние узлы дерева называют *листьями* (в нашем примере это *E, G, H, I, J*), они не имеют потомков.

Если элемент дерева не является листом, то его называют *внутренним узлом дерева*, или *вершиной*.

Пустое дерево – это дерево, не имеющее вершин.

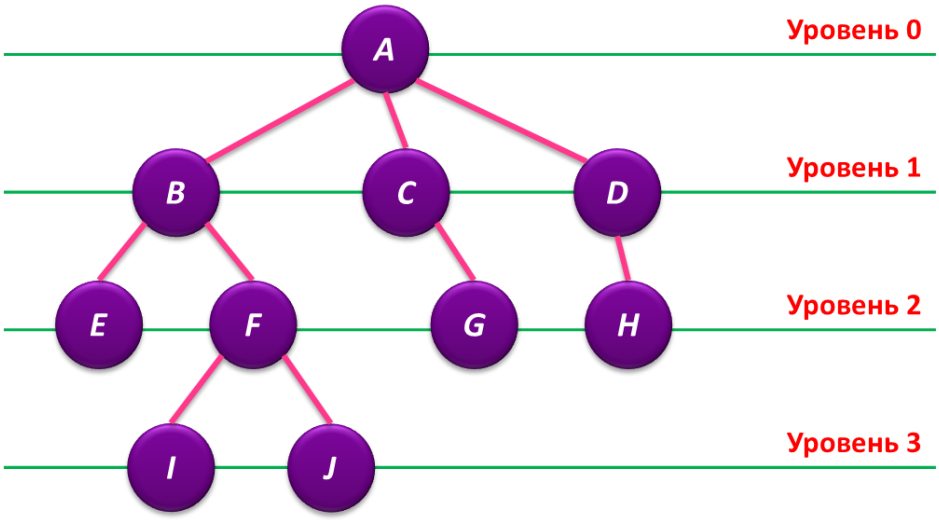


Рис. 6.9. Пример дерева

Дочерняя (сыновья) вершина – это вершина, расположенная на дереве ниже своей родительской вершины и непосредственно соединенная с ней. Например, если вершина *I* является дочерней по отношению к вершине *F*, то вершина *F* называется отцовской (материнской, родительской) по отношению к вершине *I*. Сами дочерние узлы и все их дочерние узлы называются *потомками* (узлы *E, F, I, J* – это потомки узла *B*).

Предки – это родители и прародители узла.

Каждый узел дерева является корнем поддерева, которое состоит из данного узла и всех его потомков. Например, узел *F* – это корень поддерева, содержащего узлы *F, I, J*. Узел *G* – это корень поддерева без потомков. Узел *A* – это корень поддерева, которое само оказывается деревом.

Прохождение от любого узла к его потомкам осуществляется вдоль пути. Например, путь от корня *A* к узлу *F* проходит через вершины *A, B, F*. Существует единственный путь из любого узла к его потомкам.

Уровень узла – это длина пути от корня к этому узлу. Например, уровень корня равен 0. Каждый сын корня является узлом 1-го уровня, следующее поколение – узлами 2-го уровня и т.д. Например, узел F является узлом 2-го уровня с длиной пути 2.

Глубина дерева – это максимальный уровень любого его узла. Или глубина дерева – это длина самого длинного пути от корня до узла.

Существует четыре способа изображения деревьев. Это граф, вложенные скобки, вложенные множества, ломаная последовательность. Рис. 6.1 иллюстрирует графовый способ изображения дерева. То же самое дерево можно описать, используя вложенные скобки: *(A(B(E,F(I,J)),C(G),D(H)))*. Способ изображения этого дерева вложенными множествами представлен на рис. 6.10, а ломаной последовательностью – на рис. 6.11.

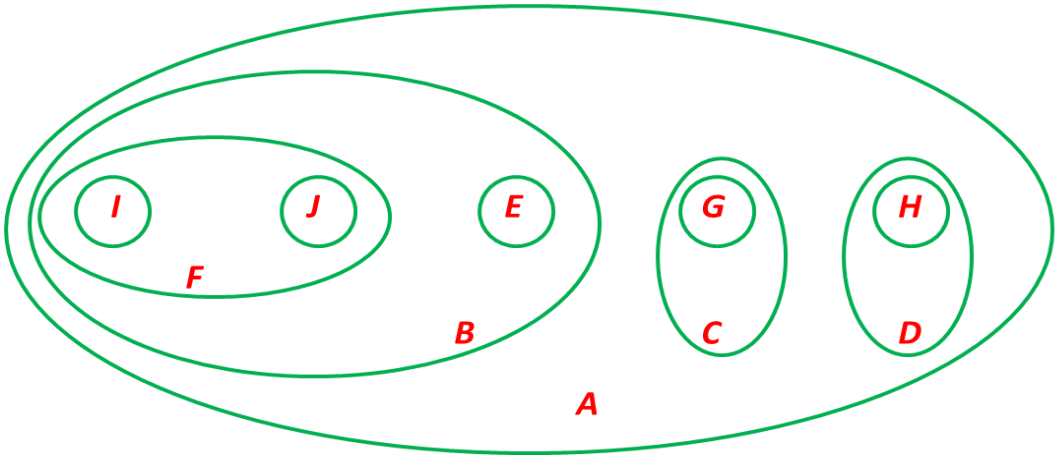


Рис. 6.10. Способ задания дерева вложенными множествами

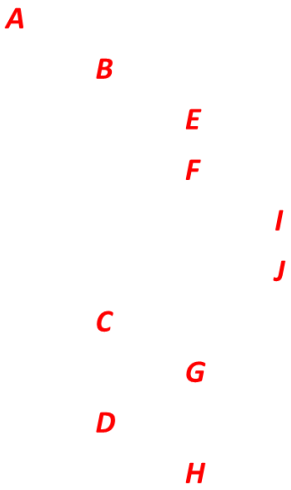


Рис. 6.11. Способ задания дерева ломаной последовательностью

Чаще всего для изображения деревьев используется граф.

6.5. Двоичные (бинарные) деревья

Двоичные деревья широко распространены в программировании. Двоичные деревья характеризуются следующими свойствами:

- в двоичном дереве каждая вершина может иметь не более двух дочерних вершин;
- для любой вершины его дочерние вершины называются правой (дочерней) вершиной и левой (дочерней) вершиной (эти названия относятся к графическому представлению дерева).

Двоичное дерево является рекурсивной структурой. Ведь каждый узел – это корень своего собственного поддерева. У него есть дочери, которые сами являются корнями деревьев, называемых левым и правым поддеревьями соответственно. Поэтому процедуры обработки деревьев чаще всего рекурсивны.

Таким образом, *двоичное дерево* – это такое множество узлов, которое разбивается на три непересекающихся подмножества:

{R} – корневой узел;
{L1, L2, ..., Lm} – левое поддерево *R*;
{R1, R2, ..., Rm} – правое поддерево *R*.

На рис. 6.12 приведен пример дерева, являющегося двоичным.

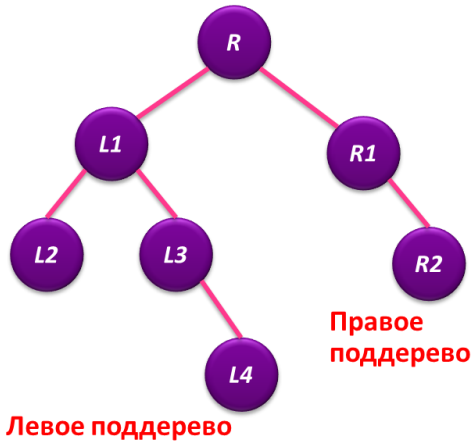


Рис. 6.12. Пример двоичного дерева

На любом уровне *n* двоичное дерево может содержать от 1 до 2*n* узлов. *Плотность дерева* – это число узлов дерева по отношению к глубине дерева. Деревья с большей плотностью важны в качестве структур данных. Плотное дерево позволяет хранить большие структуры данных и осуществлять эффективный доступ к элементам.

Вырожденные деревья и законченные деревья являются крайними мерами плотности. *Законченное двоичное дерево глубины n* – это такое двоичное дерево, у которого каждый уровень *0...n-1* имеет полный набор узлов, и все листья уровня *n* расположены слева. *Полное дерево* – это законченное бинарное дерево, которое содержит *2n* узлов на уровне *n*. На рис. 6.13 приведены примеры вырожденного, законченного и полного дерева.

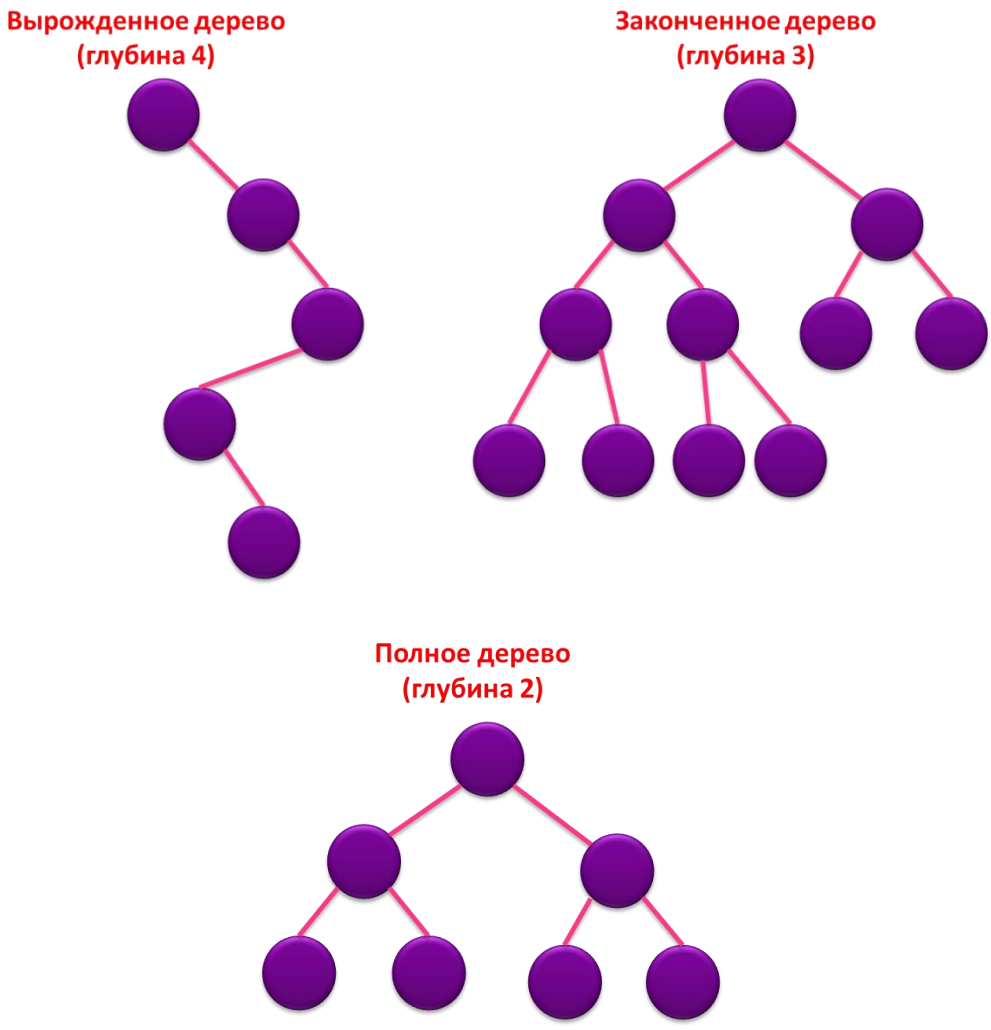


Рис. 6.13. Примеры вырожденного, законченного и полного деревьев

Как было сказано выше, глубина дерева – это длина самого длинного пути от корня к узлу. Например, для вырожденного дерева с n узлами наибольший путь имеет длину *n-1*. Тогда максимальная глубина дерева с пятью узлами равна четырем.

Для законченного дерева с *n* узлами глубина равна целой части от $log_2 n$. Например, если дерево имеет $n = 10\,000$ элементов, то глубина равна

$$\lfloor \log_2 10000 \rfloor = \lfloor 13.28 \rfloor = 13.$$

6.5.1. Структура двоичного дерева

Существует два пути представления двоичного дерева в памяти компьютера: последовательное представление с использованием обычного массива и представление в виде динамической структуры.

При последовательном представлении дерева с использованием массива двоичное дерево упаковывается в одномерный массив. Это представление использует только один линейный массив – *TREE*. Корень дерева находится в первом элементе массива *TREE[0]*, две дочерние вершины – в соседних элементах и т.д. Если узел *n* занимает элемент массива *TREE[K]*, то его левая дочерняя вершина сохранена в *TREE[2*K+1]*, а правая дочерняя вершина – в *TREE[2*K+2]*. С таким представлением дерева мы уже встречались при рассмотрении метода пирамидальной сортировки.

Данный подход имеет следующие недостатки:

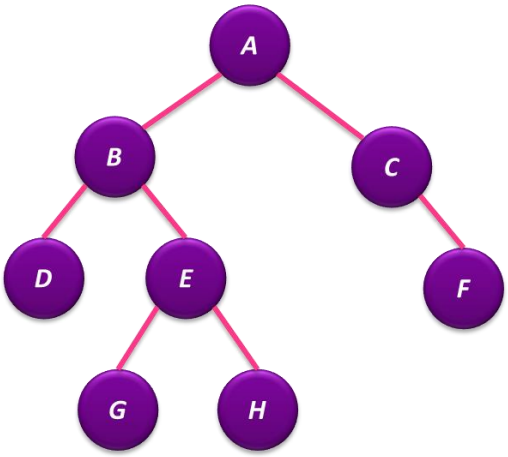
- размер дерева ограничен размером массива;
- для хранения дерева с небольшой плотностью требуется массив, большая часть которого может оказаться неиспользуемой.

Поэтому при реализации двоичных деревьев чаще всего используются динамические структуры данных.

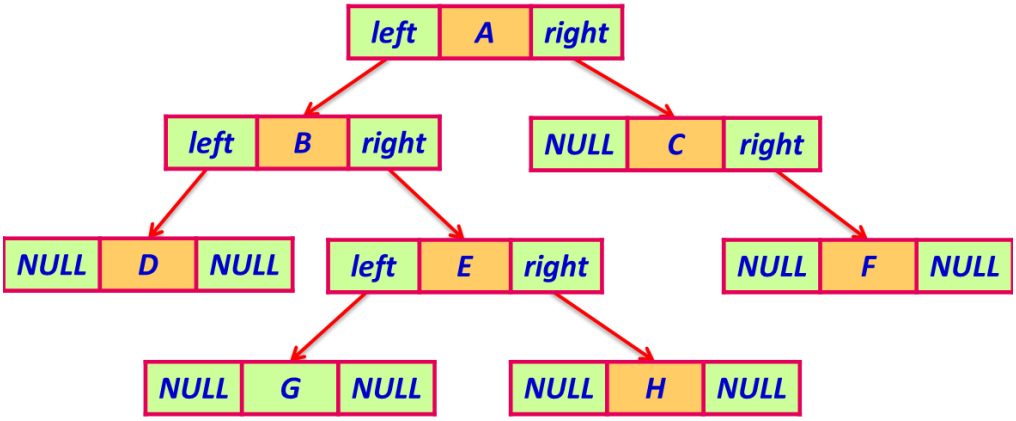
Структура дерева может быть представлена следующим образом:

Каждый узел дерева содержит поле данных и два поля с указателями. Поля указателей называются левым указателем и правым указателем, потому

что они указывают на левое и правое поддерево соответственно. Листовой узел содержит *NULL* в поле правого и левого указателей. Бинарное дерево и соответствующая ему схема структуры данных приведена на рис. 6.14 *a*) и *b*) соответственно.



a)



b)

Рис. 6.14. Структура данных для представления двоичного дерева

a) бинарное дерево;
b) схема реализации бинарного дерева в виде динамической структуры

Допустим, необходимо построить дерево с n узлами и минимальной высотой. Для этого нужно располагать максимально возможное число узлов

на всех уровнях, кроме самого нижнего. *Идеально сбалансированное дерево* – это дерево с максимально возможным числом узлов на всех уровнях, кроме самого нижнего. В идеально сбалансированном дереве для каждого узла число узлов в левом и правом поддеревьях различаются не более чем на 1.

Рассмотрим ***алгоритм построения идеально сбалансированного дерева*** при известном числе узлов *n*:

1) взять один узел в качестве корня;
2) построить левое поддерево с *nl=n/2* узлами при помощи этого же алгоритма;
3) построить правое поддерево с *nr=n-nl-1* узлами при помощи этого же алгоритма.

Пример. Для последовательности чисел – 1, 2, 3, 4, 5, 6, 7, 8, 9, 0 идеально сбалансированное дерево будет иметь вид, представленный на рис. 6.15.

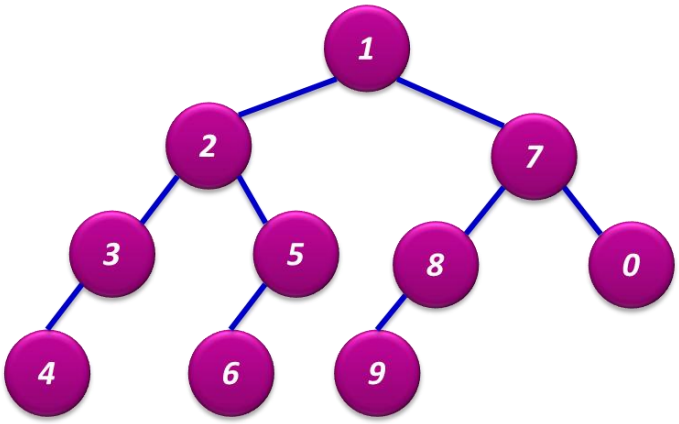


Рис. 6.15. Пример идеально сбалансированного дерева

6.5.2. Двоичные деревья выражений

Структуры типа двоичных деревьев часто применяются для представления математических выражений.

Этот тип деревьев называется деревьями выражений, потому что каждое такое дерево описывает некоторое выражение. Двоичные деревья, описывающие выражения только с двоичными операциями (двоичные операции – это операции, имеющие два операнда), называются двоичными деревьями выражений.

Например, выражение 2+3 можно описать деревом, представленным на рис. 6.16 *a*); выражение 7+(6*4) представляется деревом, представленным на рис. 6.16. *b*).

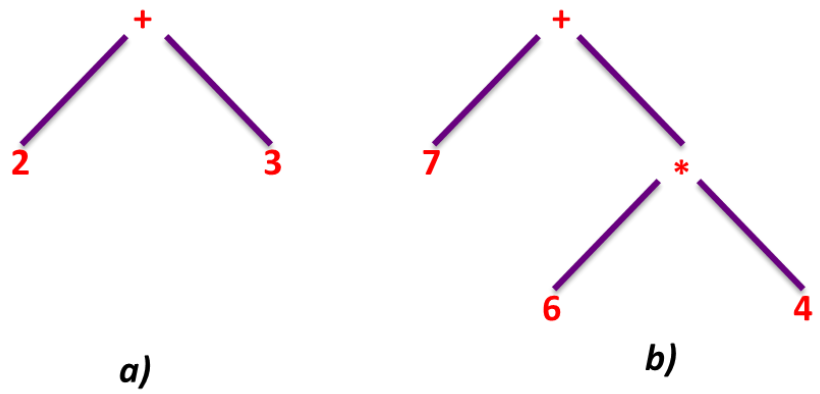


Рис. 6.16. Двоичные деревья выражений

a) двоичное дерево выражения 2+3;
b) двоичное дерево выражения 7+(6*4)

Двоичные деревья выражений имеют следующие свойства:

- каждая листьевая вершина содержит простой операнд, а каждая нелистьевая вершина содержит операцию;
- каждое поддерево представляет собой некоторое подвыражение;
- левое (или правое) подвыражение должно быть вычислено перед выполнением операции, соответствующей корню поддерева.

Деревья выражений часто используются в компиляторах и интерпретаторах для анализа семантики выражений. Обобщенное его понятие применяется в компиляторах для анализа синтаксиса программ.

6.5.3. Деревья двоичного поиска

Деревья двоичного поиска широко распространены в программировании. Значение содержимого каждой вершины дерева двоичного поиска:

1) больше или равно, чем содержимое любой из вершин его левого поддерева;

2) меньше, чем содержимое любой из вершин его правого поддерева.

Пример дерева двоичного поиска приведен на рис. 6.17.

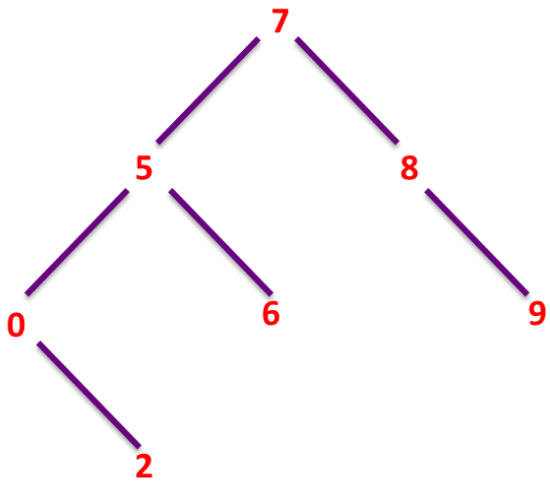


Рис. 6.17. Пример дерева двоичного поиска

Распространенность деревьев двоичного поиска в программировании является следствием эффективности методов поиска в этих деревьях. Для линейных структур временная сложность алгоритма последовательного поиска равна $O(n)$, где n – это количество элементов структуры. Для законченного бинарного дерева временная сложность алгоритма поиска равна $O(\log n)$.

Например, в списке из 10 000 элементов максимальное число сравнений при последовательном поиске равно 10 000. Поиск на законченном дереве потребовал бы не более 14-ти сравнений.

6.6. Алгоритмы обхода дерева

Алгоритм перебора вершин дерева называется алгоритмом обхода дерева. Каждый алгоритм обхода дерева выполняет в узле три действия: заходит в узел, рекурсивно спускается по левому поддереву, рекурсивно спускается по правому поддереву. Алгоритмы обхода дерева отличаются порядком, в котором они выполняют свои действия в узле.

Существует три основных метода обхода дерева:

- прямой метод;
- симметричный метод;
- обратный метод.

Прямой метод обхода дерева (сверху вниз, префиксный) предполагает следующую последовательность действий:

- посещение узла;
- прохождение левого поддерева;
- прохождение правого поддерева.

Например, для дерева, представленного на рис. 6.17 порядок посещения узлов будет следующий: 7 5 0 2 6 8 9.

Симметричный метод обхода дерева (слева направо, инфиксный) предполагает следующую последовательность действий:

- прохождение левого поддерева;
- посещение узла;
- прохождение правого поддерева.

Для этого же дерева (рис. 6.17) порядок посещения узлов будет следующий: 0 2 5 6 7 8 9.

Симметричный метод обхода дерева выводит элементы двоичного дерева поиска в порядке их возрастания.

Обратный метод обхода дерева (снизу вверх, постфиксный) предполагает следующую последовательность действий:

- прохождение левого поддерева;
- прохождение правого поддерева;
- посещение узла.

Для этого же дерева (рис. 6.17) порядок посещения узлов будет следующий: 2 0 6 5 9 8 7.

При обходе двоичного дерева выражений рассмотренными тремя способами получаем:

- ✓ Прямой метод обхода соответствует префиксному формату записи выражения.
- ✓ Симметричный метод обхода соответствует инфиксному формату записи выражения.
- ✓ Обратный метод обхода соответствует постфиксному формату записи выражения.

6.7. Вставка и удаление элемента из дерева двоичного поиска

Рассмотрим ***алгоритм вставки элемента в дерево двоичного поиска*** (просмотр дерева всегда начинается с его корня):

1. Значение, помещаемое в дерево, сравнивается со значением текущего узла.

2. Если значение, помещаемое в дерево, меньше значения текущего узла, то проверяется следующее:
   a) если у текущего узла слева нет наследников, то прикрепляем узел со значением в качестве левого наследника;
   b) иначе спускаемся по левой ветви дерева на уровень ниже.

3. Если значение, помещаемое в дерево, больше или равно значению текущего узла, то проверяется следующее:
   a) если у текущего узла справа нет наследников, то прикрепляем узел со значением в качестве правого наследника;

b) иначе спускаемся по правой ветви дерева на уровень ниже.

Алгоритм удаления элемента из дерева двоичного поиска зависит от расположения этого элемента в дереве и включает в себя четыре ситуации:

1. Элемента со значением, равным *x*, нет;
2. Элемент со значением *x* является терминальным узлом (рис. 6.18 *а*));
3. Элемент со значением *x* имеет одного потомка (рис. 6.18 *б*));
4. Элемент со значением *x* имеет двух потомков (рис. 6.18 *в*)).

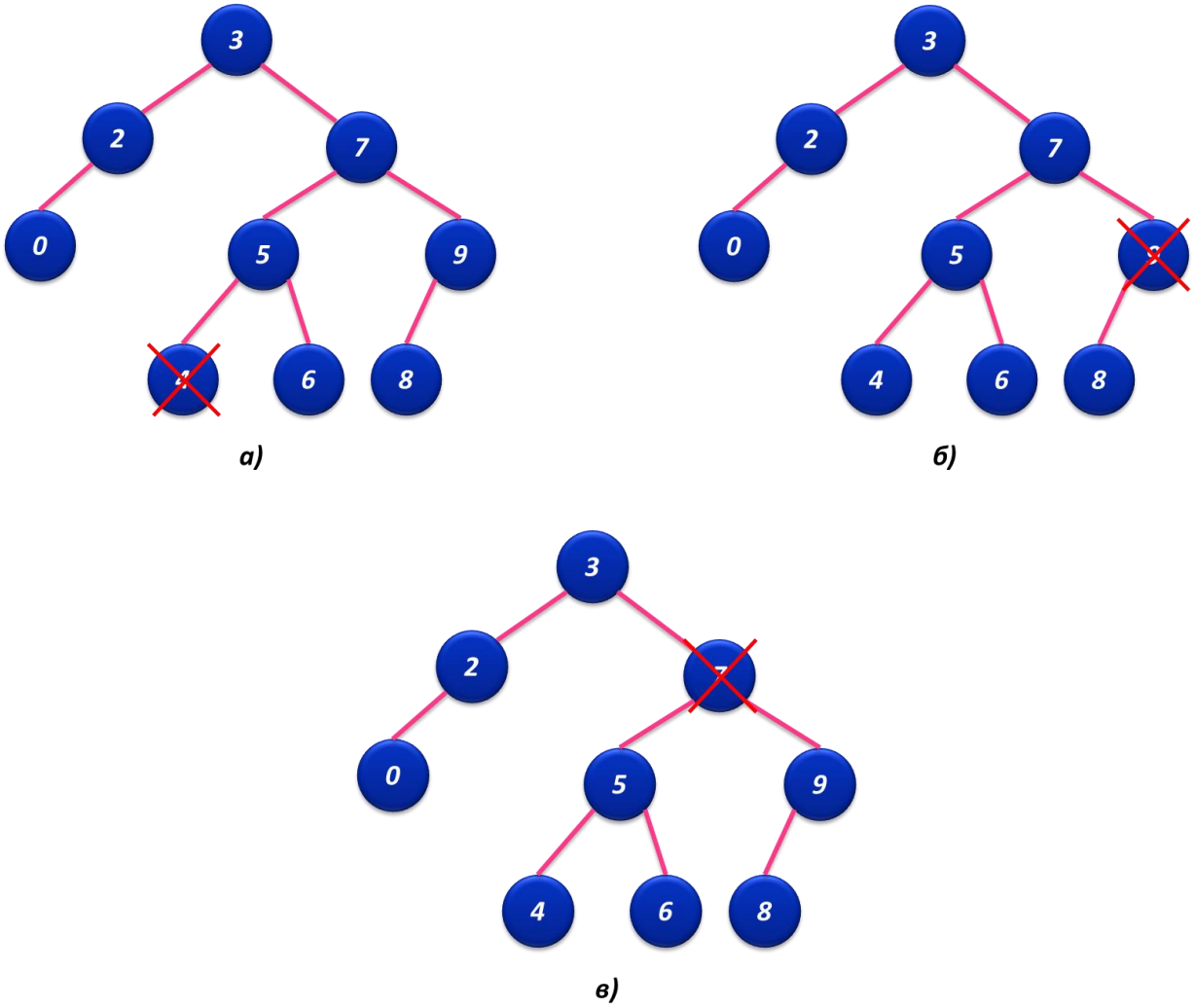


Рис. 6.18. Ситуации расположения удаляемого узла

При удалении листа или элемента, имеющего одного потомка, сложности нет. Действия аналогичны удалению элемента в линейном списке. Если элемент имеет двух потомков, то одной ссылкой нельзя указать на два направления. В этом случае удаляемый элемент необходимо заменить. Для замены выбирается самый правый элемент его левого поддерева или самый левый элемент его правого поддерева (в нашем примере на рис 6.18 *в*) элементы 6 или 8 – это ближайшие по значению к удаляемому элементу).

Рассмотрим теперь собственно алгоритм удаления элемента из дерева двоичного поиска. Алгоритм включает в себя четыре ситуации, рассмотренные выше.

Ситуация 1: Удаляемый узел не найден.

Действия: Алгоритм удаления заканчивает работу.

Ситуация 2: Узел не имеет потомков, т.е. является листом.

Действия: Обновить его родительский узел так, чтобы его поддерево оказалось пустым.

Ситуация 3.1: Узел имеет левого потомка, но не имеет правого.

Действия: Присоединить левое поддерево узла к его родителю.

Ситуация 3.2: Узел имеет правого потомка, но не имеет левого.

Действия: Присоединить правое поддерево узла к его родителю.

Ситуация 4: Удаление узла с двумя потомками.

Действия: Необходима замена удаляемого узла.

Выберем для замены самый правый узел левого поддерева. Определим следующее:

- *D* – удаляемый узел;
- *P* – родитель удаляемого узла;
- *R* – заменяющий узел;
- *PR* – родитель узла *R*.

Для нахождения узла *R* необходимо:

1. Перейти к левому поддереву узла *D*, потому что заменяющий узел *R* меньше, чем удаляемый узел *D*.

2. Заменяющий узел *R* является максимальным узлом левого поддерева узла *D*. Спускаемся по правому поддереву и ищем заменяющий узел *R*. Во время спуска следим за предшествующим узлом *PR*. При этом возможны два случая:

2.1. Правое поддерево пусто. Заменяющий узел *R* является левым сыном удаляемого узла. *PR* соответственно указывает на удаляемый узел *D*. Тогда выполняем следующие действия:

2.1.1. Присоединяем правое поддерево узла *D* в качестве правого поддерева к узлу *R*;

2.1.2. Родителя *P* удаляемого узла присоединяем к *R*.

2.2. Правое поддерево не пусто. Тогда заменяющий узел будет или листовым узлом или узлом, имеющим только левое поддерево. В любом случае выполняем следующие действия:

2.2.1. Отсоединить узел *R* от дерева.

2.2.2. Потомков узла *R* присоединить к его родительскому узлу *PR* (Левое поддерево *R* присоединяется в качестве правого поддерева к *PR*).

2.2.3. Удаляемый узел *D* заменяется узлом *R*: наследники узла *D* присоединяются в качестве наследников к узлу *R*, узел *R* присоединяется к родительскому узлу *P*.

Вопросы для повторения

1. Что такое граф? Охарактеризуйте основные понятия теории графов.
2. Опишите представление графов в памяти ЭВМ с помощью матриц смежности.
3. Опишите представление графов в памяти ЭВМ с помощью матриц инцидентности.
4. Пишите представление взвешенных графов в памяти ЭВМ с помощью матриц весов.
5. Опишите представление графов в памяти ЭВМ с помощью массивов смежности.
6. Опишите представление графов в памяти ЭВМ с помощью списков смежности.
7. Опишите представление графов в памяти ЭВМ с помощью списков ребер.
8. Какие алгоритмы обхода графов вам известны?
9. Что такое дерево? Охарактеризуйте основные понятия, связанные с деревом.
10. В чем заключается особенность двоичных деревьев?
11. Для каких целей применяются двоичные деревья? Какие виды двоичных деревьев вы знаете?
12. Как двоичные деревья представляются в памяти ЭВМ?
13. Какие способы обхода двоичных деревьев вам известны?
14. Как осуществляется вставка и удаление элемента дерева двоичного поиска?